

中山大學

SUN YAT-SEN UNIVERSITY

数据库原理实验报告

二次元周边商品店数据库重构与应用实现

姓 名　郑立峰　李维政

学 号　24325341　24325143

学 院　计算机学院

专 业　计算机科学与技术

2026 年 10 月 24 日

目录

1 实验目标与阶段安排 **1**
1.1 从关系数据库到可操作应用 . . . 1
1.2 从现货销售开始 . . . 1
1.3 实验环境 . . . 1

2 第 5 周 ER 模型与业务规则检查 **1**
2.1 实体划分与联系表达 . . . 1
2.2 商品与库存的基数和参与约束 . . . 2
2.3 销售订单与人员联系 . . . 3
2.4 预售与采购规则及原结构问题 . . . 4

3 第 6 周函数依赖与规范化取舍 **5**
3.1 候选码与主要函数依赖 . . . 5
3.2 规范化示例与分解性质 . . . 6
3.3 保留字段的设计理由 . . . 6
3.4 快照，到货数量与约束的剩余边界 . . . 7

4 迁移方案与回归结果 **8**
4.1 三项结构调整及执行顺序 . . . 8
4.2 冲突处理与实际调试 . . . 8
4.3 迁移前后语义保持 . . . 9

5 第 7–9 周应用设计与实现 **10**
5.1 从一次操作理解分层 . . . 10
5.2 查询与写入的执行过程 . . . 11
5.3 身份与三层权限控制 . . . 12
5.4 草稿与结账的事务边界 . . . 13

6 本机业务流程与结果分析 **14**
6.1 商品维护与库存入库 . . . 14
6.2 草稿编辑，结账与数据库核对 . . . 15
6.3 两位顾客的本人订单查询 . . . 16
6.4 库存不足与整单回滚 . . . 17

7 集成测试与结果分析 **18**

8 分工，AI 辅助与实验总结 **19**
8.1 小组分工与辅助过程 . . . 19
8.2 实验认识与当前不足 . . . 19

1 实验目标与阶段安排

1.1 从关系数据库到可操作应用

第一阶段已经能在 SQL Server 中保存商品，库存和订单，但从表里查到数据，还不能说明一次销售就能完整地完成。第 5–9 周沿着这个问题继续往下做，先检查原有关系能否表达业务，再调整需要补上的约束，最后让不同角色通过网页完成操作，并回到数据库核对结果。

1.2 从现货销售开始

现货销售比较适合作为这一阶段的完整流程，商品建档以后要入库，店员登记商品和数量，再确认结账，顾客最后能够查到自己的订单。这几步连在一起，价格，库存和操作身份之间的问题才会显现出来。所以应用先把这条流程做完整，原有预售和采购数据继续保留，履约和验收界面留待后续扩展。

第六周检查后保留了原有 15 张业务表和 96 个字段。网页应用需要知道当前是谁在操作，因此另外增加 `WebUser` 登录表，两个编号序列，五类受限数据库身份和顾客订单行级过滤，业务表本身的含义没有随界面开发改变。

1.3 实验环境

应用使用 Python 3.12 和 Flask，通过 pymssql 连接 SQL Server，运行环境由 Conda 管理。实验另建 `AnimeGoodsStoreDB_Week6App`，依次装入基础数据，迁移和应用安全结构。后面的操作主要在网页完成，SSMS 用来核对同一笔业务实际保存了什么。

2 第 5 周 ER 模型与业务规则检查

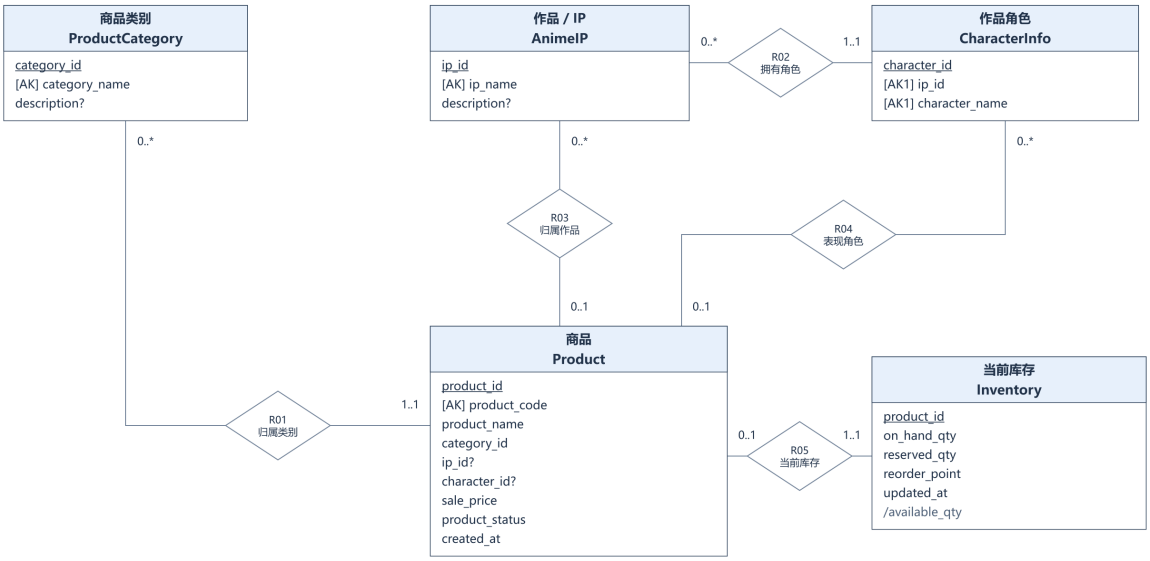
2.1 实体划分与联系表达

这次 ER 建模从已经存在的 15 张表往回检查，重点看一张表在业务中表示什么，以及它与其他记录怎样联系。商品类别，IP 和人员资料可以独立存在，一笔订单则需要通过明细才能说明买了哪些商品。按这种关系重新看表，比只按表名连线更容易发现遗漏的规则。

2.2 商品与库存的基数和参与约束

商品有时能明确对应一个角色，有时只对应一部作品。比如群像商品不适合强行填入某个角色，所以 IP 和具体角色允许按实际情况填写。指定角色以后，所属 IP 就不能任意选择，两者的组合约束用来避免把一个角色放到另一部作品下面。每件商品仍须归属一个类别。

库存以 `product_id` 同时作为主码和外码，所以一件商品至多对应一条当前库存。原结构允许商品已经建档而库存行尚未建立，网页中再遇到这种情况，就要区分没有库存记录与库存为零。应用因此在新增商品的同一事务里补建零库存行，后续入库便能从明确的零开始。



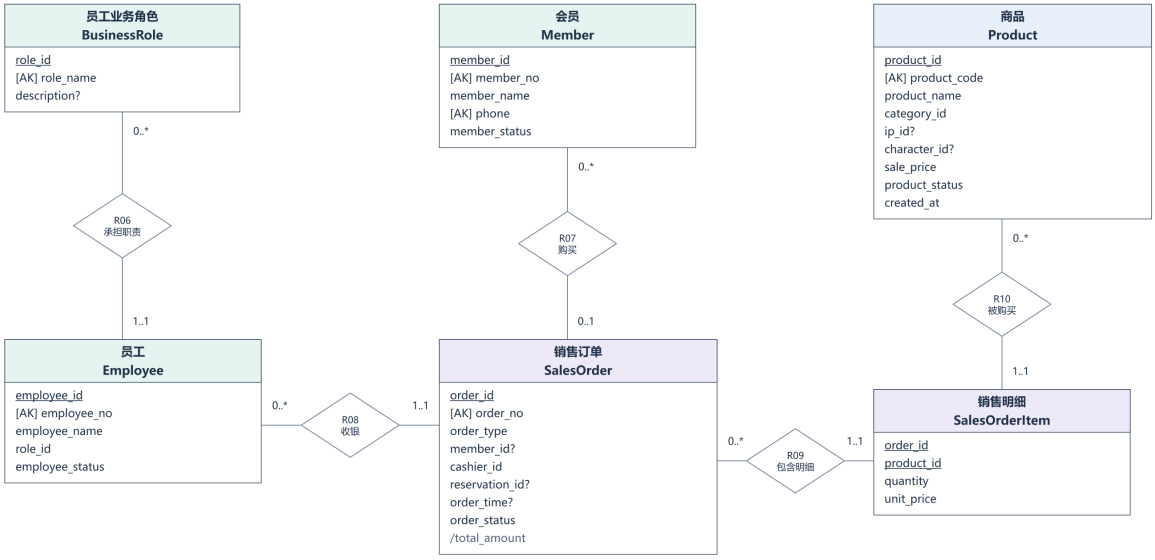
商品与库存模块 ER 图

图中用最小和最大次数标记参与联系的范围，下划线标记主码，AK 辅助标记候选码，问号和斜线分别表示可空与派生属性。这里的可售数量由实际库存减去预留量得到，查询时算出即可，另存一份反而还要在每次库存变化时同步维护。

2.3 销售订单与人员联系

销售订单先记录由谁收银，购买者可以是会员，也可以是散客。具体买了什么放在明细中，以 `order_id,product_id` 共同定位一行。同一商品再次加入同一订单时，只需修改这一行的数量，数量和登记单价也都属于这笔订单里的商品，不能只由商品编号决定。

订单与明细的关系还要结合状态理解。刚创建草稿时没有商品是正常的，到了结账时，空订单却应被拒绝。明细外码只能保证它指向的订单存在，无法独自判断订单现在能否完成。所以这条规则要继续放到结账检查中，完成后的明细也随状态变为只读。



销售，明细与人员模块 ER 图

收银员工存在，不等于当前登录者就有权完成收银。业务角色表保存员工职责，网页登录后还要核对员工是否启用，以及账户绑定的职责是否相符。这样订单中的经办身份才能与实际登录身份对应起来。

2.4 预售与采购规则及原结构问题

会员取消预订后还可能重新预订，如果把活动和会员的组合作为唯一标识，就会挡住这段需要保留的历史。因此 `presale_id,member_id` 不能直接作为预订表候选码。提货时还要对应原来的会员和商品，一条预订最多形成一笔提货订单，这些条件也需要连起来检查。

采购采用一场预售最多一张采购单的约定，普通补货则不关联预售活动。回看原结构时，几个问题正是出现在这种约定与实际约束之间，业务上已经说明了应当怎样做，表中却仍能保存不符合约定的记录。下面将它们与后续处理放在一起核对。

编号	原结构问题	后续处理与保留的问题
P01	同一预售可能关联多张采购单。	第六周为非空活动编号增加筛选唯一索引，限制同活动最多一张采购单。
P02	状态包含部分到货，与整单到货约定不一致。	删除 `PARTIAL_RECEIVED` 状态，保留明细到货数量，数量与整单状态的联动仍未强制。
P03	完成订单可能无明细或继续被修改。	应用允许草稿编辑，结账拒绝空单，完成后交易内容只读。
P04	商品建档后可能没有库存行。	应用新增商品时同事务创建零库存行，库存调整也处理缺失记录。
P05	普通补货可能错误关联预售活动。	第六周以双向 CHECK 同时约束采购类型和活动编号。
P06	预售会员，商品，预订状态存在跨表规则。	留待完整预售履约接口处理。
P07	销售，库存与操作身份可能脱节。	当前现货结账通过登录身份，角色检查及统一事务实现，采购和预售链尚待扩展。

表 1: 由 ER 反查得到的问题与落实位置

这些问题有的能直接变成唯一约束或 CHECK，有的却要等到一次业务操作中同时检查几张表。ER 图把关系整理出来以后，接下来还要判断规则应落在哪里，才能从图上的约定走到实际能够拒绝错误数据的实现。

3 第 6 周函数依赖与规范化取舍

3.1 候选码与主要函数依赖

样例中某列没有重复，还不足以说明它就是候选码。这里先根据业务约定和唯一约束确认哪些属性能标识一条记录，再检查其他属性依赖单个码，整组码，还是其中某个非主属性。这样才有依据讨论部分依赖和传递依赖。

关系	主要候选码	依赖与分析结论
`ProductCategory`	类别 ID，类别名称	任一候选码确定描述等属性，类别名称单独存储，避免在商品行内反复更新。
`CharacterInfo`	角色 ID，IP 与角色名称组合	角色 ID 确定 IP 及角色名称，辅助唯一组合 `character_id,ip_id` 是超码，但不是最小候选码。
`Product`	商品 ID，商品编号	商品码确定商品资料，在指定角色的子集内还存在角色 ID 决定 IP 的依赖，保留受控冗余。
`Inventory`	商品 ID	商品 ID 确定当前数量，预留数量，警戒值和更新时间，可售数量由查询计算。
`SalesOrder`	订单 ID，订单编号	订单码确定会员，收银员及状态，非空预订编号的条件唯一性不作为全表普通候选码。
`SalesOrderItem`	订单 ID 与商品 ID 组合	整组码确定数量和成交单价，不将商品当前售价误作历史成交价的决定因素。
`PresaleReservation`	预订 ID，预订编号	保留取消与重订历史，活动与会员组合不保证全表唯一。
`PurchaseOrderItem`	采购 ID 与商品 ID 组合	整组码确定采购量，成本和到货量，数量约束与状态联动需要分别判断。

表 2: 代表性关系的候选码与依赖分析

例如会员手机号有唯一约束，可以据此分析候选码，员工手机号和供应商名称则不能照搬这个结论。再看订单明细，商品名可以由商品编号确定，但买了几件，以什么单价登记，还必须知道是哪一笔订单。所以商品资料单独保存，数量和单价留在明细中。

3.2 规范化示例与分解性质

为了说明现有商品表与类别表为什么分开，先在理论上把它们合成一个关系。

$$R(\mathit{product_id}, \mathit{product_name}, \mathit{category_id}, \mathit{category_name})$$

商品 ID 决定商品名与类别 ID，类别 ID 又决定类别名。如果每件商品都带着一份类别名称，改一次类别名就要改多行。还没有商品的类别难以单独插入，最后一件商品被删除时，类别本身也可能跟着丢失，更新异常便由这种重复保存产生。

把商品资料与类别资料分开以后，得到下面两个关系。

$$R_1 = (\mathit{product_id}, \mathit{product_name}, \mathit{category_id})$$
$$R_2 = (\mathit{category_id}, \mathit{category_name})$$

两个关系共有类别 ID，而类别 ID 能够决定 R_2 的全部属性，所以按类别 ID 连接时，可以把商品与它所属的类别重新对应起来，这个分解是无损的。原来的两组依赖分别留在 R_1 和 R_2 内部，检查它们也不需要先把表连接回来，因此依赖得到保持。

这个合并关系只用于分析。原库中的商品和类别已经分开，第六周无需再执行一次相同的拆分。规范化检查给出的结果是继续保留这部分结构，实际迁移再去处理采购约束中确实存在的问题。

3.3 保留字段的设计理由

商品同时保留 IP 和具体角色。从角色能够查到所属 IP，看起来商品中的 IP 可以省去。但群像商品只关联作品，不一定对应某个具体角色，删去 IP 就失去了这部分表达。另一方面，在角色非空的记录中，角色 ID 决定 IP，角色 ID 却不是商品表超码，IP 也不是主属性，这部分条件依赖不满足严格第三范式。这里保留两个字段，用组合外码检查它们是否匹配，同时承认需要维护这份冗余。

保留历史值，派生值尽量查询计算。商品现在卖多少，与某笔订单按多少成交，是两个时点的问题。明细单价要留下本次购买的价格，目录以后调价也不能改掉这段记录。总额则由明细数量乘单价求和，可售量由实际库存减预留量得到，查询时算出就能与当前记录一致，不必再保存一份等待同步的数值。

3.4 快照，到货数量与约束的剩余边界

预售需求也有类似的时点区别。决定采购时参考的需求量需要留下来，之后有人取消或重订，当前需求会变，但不应覆盖当时作出决定的依据。计划采购量还可能考虑已有库存，所以它也不一定等于需求快照加额外备货量。

订金实收和退款额也按实际收付保留。如果以后明确规定实收额始终等于数量乘固定订金单价，才需要进一步讨论这部分可计算冗余。目前尚未把这种假设写成完整约束，范式分析也就保留它的适用条件。

`received_qty` 则需要回到当前真正实现的约束来看。若整单收货和明细状态已经严格同步，到货量有可能由状态与采购量推导出来。但现在只限制它处于零到采购量之间，尚未保证每行与整单状态联动。直接删掉这列，等于先假定了还没有完成的流程，因此这次仍保留到货量。

选择	采用理由	还需保证什么
保留商品 IP 与角色	支持群像商品及角色商品的不同表达。	校验角色归属 IP，避免不匹配。
保留明细成交价	保存交易历史，允许之后调整商品售价。	登记时保存目录标价，结账沿用，完成后禁止修改。
查询计算总额与可售量	数值由现有记录确定，避免重复维护。	查询与页面使用一致计算口径。
保留需求决策快照	保存采购决策时依据，便于追溯。	区分快照与当前汇总值。
暂保留采购到货量	保持原数据与操作含义。	后续实现数量与收货状态的事务联动。

表 3: 结构保留与受控冗余的取舍

所以这次保留 15 张业务表，把实际调整集中在采购规则上。保留一列同样需要说明理由，规范化分析既要指出依赖，也要看删改之后还能不能表达原来的业务。

4 迁移方案与回归结果

4.1 三项结构调整及执行顺序

先留一份未迁移的基线，再在相同数据的副本上执行调整，后面才能对照变化。脚本检查旧结构与冲突记录，满足前提后才在一个事务中建立索引和替换 CHECK，冲突时列出记录并停止，保留原数据供核对。

```
CREATE UNIQUE INDEX UX_PurchaseOrder_PresaleID
ON dbo.PurchaseOrder(presale_id)
WHERE presale_id IS NOT NULL;

ALTER TABLE dbo.PurchaseOrder WITH CHECK
ADD CONSTRAINT CK_PurchaseOrder_TypePresale CHECK
((purchase_type='PRESALE' AND presale_id IS NOT NULL)
 OR (purchase_type='RESTOCK' AND presale_id IS NULL));

ALTER TABLE dbo.PurchaseOrder WITH CHECK
ADD CONSTRAINT CK_PurchaseOrder_Status CHECK
(purchase_status IN ('CREATED','ORDERED','RECEIVED','CANCELLED'));
```

普通补货没有活动编号，唯一索引因此只检查非空的 `presale_id`，多张补货单仍可保存。已取消的采购单也受它限制，所以同一活动最多一张采购记录，取消后另建采购单仍会被拒绝。旧约束移除与回滚由完整迁移脚本处理。

采购类型要从两个方向检查，预售采购需要活动编号，普通补货则应为空，只约束前者仍会留下错误关联。状态域同时移除部分到货，但允许哪些状态，与到货数量怎样和状态一起变化，仍是两个需要分别处理的问题。

建立约束时使用 `WITHCHECK` 检查既有行，脚本同时启用 `XACT_ABORT` 和筛选索引所需选项。旧结构不符或已迁移过时就停止，避免在错误前提下改表。

4.2 冲突处理与实际调试

同活动多单，类型关联不符和非法状态都会使迁移停止，原记录仍保留，已迁移的副本再次执行则返回 51005。调试中还实际遇到过 1934 错误，原因是筛选索引要求的会话选项不一致。补齐 SET 并关闭 `NUMERIC_ROUNDABORT` 后，45 项回归通过。索引建立以后，后续写入仍受这些选项影响，所以应用连接也沿用相同设置。

4.3 迁移前后语义保持

回归分类	通过数	检查内容
DATA	15	全部业务表的行数及全列双向 EXCEPT，检查数据是否增减或改变。
QUERY	5	订单明细，金额，销量，库存等关键查询，检查业务结果是否保持。
META	7	旧结构与目标结构的键，索引，约束等元数据。
TARGET	1	新增筛选唯一索引和两项 CHECK 均启用且受信任。
WRITE	4	合法写入仍然可以执行。
REJECT	7	重复活动采购，类型不符，非法状态等反例被拒绝。
ROLE	6	既有员工角色的允许与拒绝行为。
合计	45	45 项通过，0 项失败。

表 4: 第六周独立副本回归结果

核对项目	迁移前	迁移后	解释
SO004 金额	124.00	124.00	两条明细汇总保持。
6 笔订单金额合计	530.00	530.00	关键查询语义保持。
6 件商品可售库存合计	42	42	数量为 8，5，2，12，6，9。

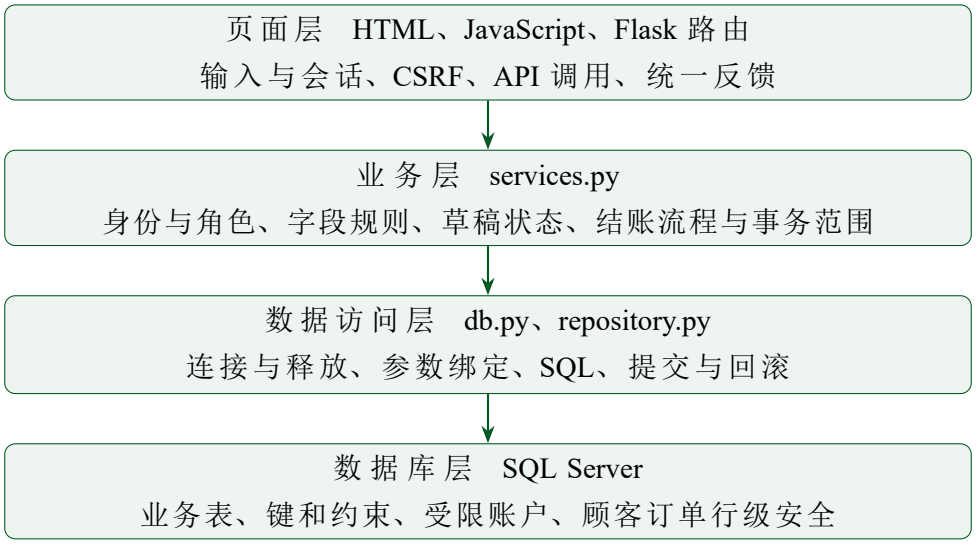
表 5: 固定样例的迁移前后对照

SO004 仍为 124 元，6 笔订单合计 530 元，可售库存合计 42 件，这些结果先说明常用查询没有因迁移而改变。再结合全部表的全列双向 EXCEPT，以及合法写入和非法输入的测试，才能进一步判断原数据得到保留，新规则也确实起作用。行数相同只是其中一部分依据。

5 第 7–9 周应用设计与实现

5.1 从一次操作理解分层

点下确认结账时，页面能够检查是否填了数量，却无法独自判断库存是否还够，订单是否已经完成。因此页面只把操作交给后端，业务层决定能不能做，数据访问层再执行 SQL 并管理连接。这样同一条规则可以在各个入口共用，不会只在某个按钮里生效。



应用分层及请求流向

模块	负责内容
`app/templates/index.html`	页面结构，表格与对话框。
`app/static/app.js`	查询，表单提交，角色入口和错误展示。
`app/__init__.py`	路由，会话，CSRF，统一响应与异常映射。
`app/services.py`	权限，输入与业务状态校验，组织完整业务操作。
`app/db.py`	建立同事务连接，提交，回滚和关闭。
`app/repository.py`	参数化 SQL 与结果读取。
`sql/01_web_security.sql`	登录技术表，受限角色与顾客行级过滤。

表 6: 各层在源代码中的位置

5.2 查询与写入的执行过程

以搜索商品为例，页面将输入的名称或编号交给 Flask 路由，后端先确认登录身份和搜索词长度，再让 Repository 查询商品与库存。结果返回后才刷新表格。搜索条件经过这条路径进入数据库，而不是由页面自己组成一段 SQL。

```
# services.py
with self.connection(user) as repo:
    return repo.products("%" + term + "%", user["role"] in STAFF)
```

```
-- repository.py 中查询的参数绑定形式
WHERE (p.product_name LIKE %s OR p.product_code LIKE %s)
```

这里的百分号和搜索词拼成 LIKE 的参数值，SQL 语句本身仍然固定。输入中即使带引号，也只会作为这个参数的一部分处理。商品名，数量和价格等写入值同样绑定为参数，查询与修改没有各自采用一套拼接办法。

新增商品比查询多了一层需要一起完成的修改。资料通过编号，类别，角色归属和价格检查后，商品与零库存行要在同一事务里保存，失败时一起撤销。删除则要再看库存和历史引用，已经参与交易的商品用停售保留记录，避免历史订单失去原来的商品关联。

库存调整接收商品编号，增减量和警戒值，保存前检查调整后的实际量不能低于预留量。草稿则接收会员，商品，数量和首次登记时看到的标价，结账仍要按当前身份重新核对订单状态。查询返回的是可见记录，写入返回成功以后还会重新读取结果，页面才能显示实际保存的状态。

这些操作的失败也由后端统一接收并转为页面提示。数量不足时保留草稿，店员可以调整后重试，连接失败则提示暂时不可用。响应与日志共用请求编号，后端另外保存异常细节，页面只显示当前需要采取的处理。

5.3 身份与三层权限控制

网页登录后先确定账户对应的员工或会员，再核对它是否启用以及承担什么职责。这个身份由会话和数据库绑定共同确定，页面不能自己选择成为店长。密码也只保存哈希，业务操作沿用已经核对过的身份。

角色	商品	库存	订单
店长 `manager`	查询，新增，修改，条件删除。	查询与调整。	创建草稿，编辑明细，结账，取消及草稿删除。
店员 `clerk`	查询。	在目录查看可售量。	创建草稿，编辑明细，结账，取消及草稿删除。
库存员 `inventory`	查询。	查询与调整。	不开放订单和会员查询。
顾客 `customer`	查询在售商品。	仅随目录查看可售量。	查询本人订单，不开放经营写入。

表 7: 四类业务角色的主要操作范围

界面初版把商品目录和库存管理都提供给了多个角色，两处却展示相近的内容，入口增加了，操作目的并没有分清。逐角色检查后，店长的商品页只维护资料与售价，库存页再处理数量，店员在目录查看可售量并进入收银，库存员则只保留库存管理。顾客不需要店铺经营统计，只浏览在售商品和本人订单。后端仍逐项检查权限，再选用 `auth`，`manager`，`clerk`，`inventory` 或 `customer` 受限身份连接数据库，页面上的简化没有代替这些检查。

两个顾客账号的角色相同，能查到的记录却应不同。所以顾客权限还要带上绑定的会员编号，连接中设置只读 `member_id` 上下文，再由 SQL Server 的行级安全策略过滤订单和明细。API 也按会员筛选，后面的测试会分别检查页面范围和跨会员直接请求。

页面校验能及时提示输入问题，后端仍要再检查一次。价格按十进制处理并限制正值，两位小数和上限，数量则拒绝零，负数，布尔值和非整数，写请求同时校验 CSRF。这样绕过表单直接提交请求时，仍然经过同样的规则。

5.4 草稿与结账的事务边界

订单先存为 CREATED 草稿，允许调整会员和数量，暂不扣库存。价格却不能因为改数量而跟着改变。原实现会在改数量和结账时取最新售价，但顾客是先看标价再决定购买，确认前后的金额应当一致。所以改为首次登记时核对并保存目录标价，已有明细的刷新，改数量和结账都沿用它。尚未登记时若页面价格已过期，就提示刷新核对，拒绝按另一个价格直接写入。结账成功时转为 COMPLETED 并保留记录。

结账时先锁定订单，确认它仍是可修改的草稿，会员有效且已经有明细，再按商品 ID 顺序检查在售状态与可售库存。关键是让所有明细共用一个事务。若每扣完一件就提交，后面的商品不足时，前面已经提交的扣减便无法随整单一起撤销。

```
UPDATE dbo.Inventory
SET on_hand_qty = on_hand_qty - %s,
    updated_at = SYSDATETIME()
WHERE product_id = %s
  AND on_hand_qty - reserved_qty >= %s;
```

库存是否足够也放进 UPDATE 的条件里，只有可售量满足数量时才扣减，再根据影响行数判断结果。这样读取库存以后即使还有其他订单参与竞争，写入时仍会重新受这个条件限制。商品锁按固定顺序取得，配合事务处理并发争用。

```
# db.py 中事务管理的核心结构
try:
    # connection 已建立，cursor 已设置所需 SET 选项
    yield Repository(cursor)
    connection.commit()
except BaseException:
    connection.rollback()
    raise
finally:
    connection.close()
```

后面的商品不足时，前面已经执行的扣减仍留在同一事务里，因此可以一起回滚，订单继续保持草稿，登记单价也不变。这里真正需要一起完成的是整笔销售，不能只看某条 UPDATE 是否执行成功。业务拒绝，约束错误和执行异常最终都沿这条回滚路径返回，连接在最后关闭。

6 本机业务流程与结果分析

6.1 商品维护与库存入库

这次先由店长新增 DEMO001，再把名称改为初音未来徽章，售价改为 120 元，类别，IP 和角色分别设为徽章，初音未来和初音未来。另一件 DEMO002 的售价为 20 元，名称实际保存成文本 None。保存后再回到目录查询，核对页面确实读到了修改后的资料，新增商品的库存则从零开始。



商品新增与修改表单，局部截图

接着换库存管理员入库，两件商品分别增加 10 件和 5 件，警戒值设为 3 和 2，查询得到可售量合计 15。商品建档与入库在这里是两步，库存员可以处理数量，却不能顺便修改商品售价或查看订单。店员需要的只是目录中的售价和可售量。



入库后两件商品库存为 10 和 5，局部截图

6.2 草稿编辑，结账与数据库核对

店员为会员 M001 建立 WEB0000010015 草稿，加入两件 DEMO001 和一件 DEMO002，金额为 260 元。先把第一行改为 1 件并保存，金额随之变成 140 元，再恢复为 2 件。这一步同时检查数量修改与重新汇总，草稿期间库存仍保持原数，最后才确认结账。

确认以后，订单变为已完成，明细不能再编辑。两行金额分别为 240 元和 20 元，总额 260 元，库存则从 10 和 5 变成 8 和 4，正好分别减少 2 件和 1 件。页面上的金额与数量变化能够对应起来。



结账后保存成交价格与订单明细，局部截图

页面显示成功之后，再到 SSMS 查询同一笔订单。关联商品，库存和明细后，状态为 COMPLETED，会员 ID 为 1，数量与单价也都和网页一致，说明这些变化已经写进数据库，而不只是页面上的提示。

Results Messages

	product_code	product_name	on_hand_qty	reserved_qty	available_qty
1	DEMO001	初音未来徽章	8	0	8
2	DEMO002	None	4	0	4

	order_no	order_status	member_id	product_code	quantity	unit_price	line_amount
1	WEB0000010015	COMPLETED	1	DEMO001	2	120.00	240.00
2	WEB0000010015	COMPLETED	1	DEMO002	1	20.00	20.00

数据库中的库存与成交明细，局部截图

6.3 两位顾客的本人订单查询

登录 `customer1`，列表中能看到刚完成的 260 元订单，以及 SO004 和 SO003。它可以打开明细核对，却不能修改商品，库存或订单，收银完成后留下的记录以只读形式提供给顾客。



顾客 1 可以查看会员 1 的订单，局部截图

换成 `customer2` 后，列表只有会员 2 的 SO002，金额 90 元，刚才那笔 260 元订单不再出现。两个账号使用同一套顾客界面，结果却随绑定会员改变，这里区分的是本人记录的范围，而不只是有没有进入订单页的权限。跨会员直接访问还会在后面的集成测试中检查。



顾客 2 只显示会员 2 的订单，局部截图

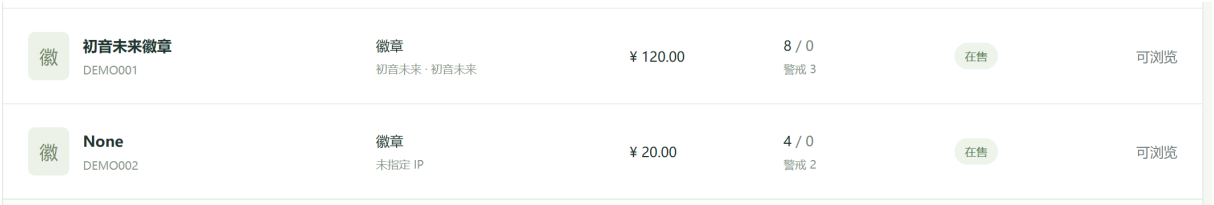
6.4 库存不足与整单回滚

接着另建 WEB0000010016，第一行买 1 件 DEMO001，第二行买 10 件 DEMO002，金额 320 元。库存分别为 8 和 4，所以第一行足够，第二行不足。特意让不足出现在后面，才能检查前面已经处理的商品是否也会撤销。确认后返回 INSUFFICIENT_STOCK 和 409，页面提示库存不足，订单仍是草稿。



库存不足时拒绝结账并保留草稿，局部截图

再查库存，结果仍是 8 和 4。结账按商品 ID 顺序处理，第一件本来可以扣减，但第二件失败后，它的扣减也被撤销了。这里比单独验证一件商品不足更能说明事务范围，失败不会留下部分商品已扣减而订单尚未完成的状态。日志也能按页面上的请求编号找到这次操作。



失败后两件商品库存保持 8 和 4，局部截图

这笔失败草稿随后确认删除，再查询时该订单编号已不存在。草稿还没有形成成交记录，可以修改后重试或删除，完成订单则保留原交易内容。其他删除条件，引用冲突和越权情况由自动化测试继续检查。

7 集成测试与结果分析

最终执行 pytest 得到 48 项通过，0 失败，0 错误，0 跳过。其中 42 项连接真实 SQL Server 检查应用行为，另外 6 项检查密码重设辅助脚本。

测试范围	代表性检查	结果
登录与请求保护	登录退出，未登录访问，密码哈希与后端身份，CSRF 与非法 JSON。	通过
商品与库存	商品 CRUD，自动零库存，重复约束回滚，非法价格，预留保护，有库存和有引用商品删除拒绝。	通过
订单与事务	草稿及明细 CRUD，空单拒绝，目录标价核对，草稿锁价，多商品结账，历史价保持，库存不足整单回滚。	通过
角色与顾客隔离	API 越权拒绝，顾客跨会员访问与 SQL RLS，受限 SQL 账户直接执行拒绝。	通过
并发与故障	并发结账不超卖，数据库连接失败与日志，页面安全响应头。	通过
密码重设辅助	生成哈希匹配且不含明文，非法输入拒绝，已有输出文件保留。	通过

表 8: 本机 48 项测试的范围与结果

价格规则另外做了两种检查。商品按 25 元登记以后把目录调为 30 元，刷新和改数量仍保留 25 元，两件结账 50 元，库存由 10 减到 8。还没登记的商品若使用过期页面标价，则由集成测试确认拒绝写入，提示重新核对。前者保留已登记价格，后者避免新登记时无提示换价，处理时点并不相同。

参数化查询直接使用带引号和 SQL 片段的搜索词，返回空列表以后，商品仍可正常查询。这个输入没有作为 SQL 执行，再结合 Repository 的参数绑定方式，检查查询条件与语句结构确实分开。

权限测试则绕过隐藏按钮，直接让顾客新增商品，店员调整库存，库存员查询订单，均得到拒绝。再用受限 SQL 账户执行越权语句，返回 229 错误。顾客跨会员访问同时检查 API 和数据库过滤，所以页面列表正确只是其中一项观察。

断连测试把连接指向不可用端口，页面得到 503，并能按请求编号找到日志。Windows 复跑时曾在读取这份日志处失败，实际是 UTF-8 日志被按默认编码读取，明确读取编码后全量复跑 48 项通过。这里需要区分连接失败已经正确处理，与测试读取记录本身出了问题。

并发用例设置实际库存 3 件，预留 1 件，让两个订单各买 2 件，结果只有一笔成功，另一笔返回库存不足，最终实际库存与预留量均为 1。这个结果验证了所设置的争用场景，没有把两次购买都算作成功，尚不能代替更大规模的压力测试。

8 分工，AI 辅助与实验总结

8.1 小组分工与辅助过程

郑立峰负责第 5–6 周的 ER 模型，规则检查，依赖与范式分析，以及采购约束迁移和回归材料。李维政在这个结构上完成第 7–9 周应用，连接数据库，区分角色操作，处理结账事务与异常，并进行集成测试和本机流程截图，再将各周结果整理进报告。最终提交前仍需共同核对图，代码和实际结果是否一致。

AI 参与了规则分析，代码和测试组织，连接排查，以及报告的整理。业务范围和字段取舍由小组确认，本机截图记录实际的角色切换，数量修改，结账和失败操作。自动测试的运行结果另行保留，报告里的价格，金额与库存都对应已有截图或查询结果，未用预设演示值替换实际观察，过程见 `ai_log.md`。

8.2 实验认识与当前不足

回看这一阶段，表结构和完整流程之间还有不少需要补上的判断。外码能保证明细找到订单，却不能判断空订单能否结账，角色存在也不等于当前账户就有收银权限。把这些关系真正放进一次操作以后，才能分清哪些交给数据库约束，哪些需要由业务层继续处理。

价格规则的修改也让历史值的含义更具体了。目录价说明商品现在怎样定价，登记单价说明这笔购买接受了什么价格，两者在不同时间发生变化并不矛盾。已有明细需要保留约定，新登记又要核对页面所见价格，单纯在结账时取一次最新售价，反而会破坏确认前后的对应关系。

库存失败案例则给出了更直接的核对方式，第一件够卖，第二件不足，最后两件库存都没有变化，才能说明整笔销售确实回到了原来的状态。一次成功扣减本身看不出这个区别，所以测试中仍要保留会失败的输入，并核对失败之后留下了什么。

现货流程目前已有对应的页面，数据库结果和测试依据。预售履约与采购验收还没有沿同样方式完成，到货数量和整单状态的联动也仍需继续实现，这部分不能从当前现货测试的通过结果直接推出。